

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250277900

Kind Code

A1

Publication Date

September 04, 2025

Inventor(s)

McLaughlin; Michael et al.

RANGING WITH SIMULTANEOUS FRAMES

Abstract

Embodiments of a device for wireless communication are disclosed. In some embodiments, the device includes: a transceiver configured to receive a message from a second device and at least one processor. The processor is communicatively coupled to the transceiver and is configured to: generate a response message includes a SYNC portion, a start frame delimiter (SFD) portion, and a cipher portion. The SYNC portion includes a sequence of symbols forming a preamble. The cipher portion includes a ciphered sequence of pseudo-randomized pulses. The transceiver is further configured to transmit the response message.

Inventors: McLaughlin; Michael (Dublin, IE), Niewczas; Jaroslaw (Jozefow, PL)

Applicant: Qorvo US, Inc. (Greensboro, NC)

Family ID: 61007241

Appl. No.: 19/196416

Filed: May 01, 2025

Foreign Application Priority Data

GB	1720550.1	Dec. 08, 2017
GB	1804735.7	Mar. 23, 2018
GB	1811245.8	Jul. 09, 2018

Related U.S. Application Data

parent US continuation 18439013 20240212 PENDING child US 19196416

parent US continuation 18310810 20230502 parent-grant-document US 12228634 child US 18439013

parent US continuation 16761160 20200501 parent-grant-document US 11681034 WO

continuation PCT/EP2018/084058 20181207 child US 18310810

Publication Classification

Int. Cl.: **G01S13/76** (20060101); **B60R25/20** (20130101); **B60R25/24** (20130101); **B60W30/06** (20060101); **G05D1/00** (20240101); **G05D1/226** (20240101); **H04B1/7163** (20110101); **H04W12/037** (20210101)

U.S. Cl.:

CPC **G01S13/765** (20130101); **H04B1/7163** (20130101); **H04W12/037** (20210101); **B60R25/2072** (20130101); **B60R25/24** (20130101); **B60R2325/108** (20130101); **B60W30/06** (20130101); **G05D1/226** (20240101); **H04B2201/71634** (20130101)

Background/Summary

RELATED APPLICATIONS [0001] This application is a continuation of U.S. patent application Ser. No. 18/439,013, filed Feb. 12, 2024, which is continuation of U.S. patent application Ser. No. 18/310,810, filed May 2, 2023, now U.S. Pat. No. 12,228,634, which is a continuation of U.S. patent application Ser. No. 16/761,160, filed May 1, 2020, now U.S. Pat. No. 11,681,034, which is a national stage entry of and related to International Application Serial No. PCT/EP2018/084058, filed Dec. 7, 2018, the disclosures of which are hereby incorporated herein by reference in their entireties.

FIELD OF THE DISCLOSURE

[0002] This disclosure relates generally to communication systems and methods of operating the same.

BACKGROUND

[0003] The present disclosure relates to systems and methods for communication between devices, where at least one of the devices includes multiple transceivers. Particular examples relate to ranging exchanges between such devices.

[0004] Provision of multiple transmitters on a device (especially larger devices such as vehicles) can improve the likelihood that transmissions will be reliably received at other remote devices (e.g., a wireless keyfob for the vehicle). Such transmissions may be used e.g., for ranging (distance determination) or remote control (e.g., door locking/unlocking, automatic parking functions, etc.) However, the need for the remote device to listen for communications from any of the transmitters increases power requirements at the remote device, which is problematic for many small devices (e.g., keyfobs) which typically have limited battery capacity and/or long battery life requirements.

[0005] Embodiments of the disclosure seek to ameliorate such problems and provide improved communication schemes for multi-transmitter devices.

SUMMARY

[0006] In some embodiments, a device for wireless communication includes: a transceiver configured to: receive a message from a second device; at least one processor communicatively coupled to the transceiver, the at least one processor configured to: generate a response message includes a SYNC portion, a start frame delimiter (SFD) portion, and a cipher portion, wherein: the SYNC portion includes a sequence of symbols forming a preamble; and the cipher portion includes a ciphered sequence of pseudo-randomized pulses; the transceiver further configured to: transmit the response message. In some embodiments, the processor is configured to generate the cipher portion using a pseudo-random number generator. In some embodiments, the message further includes a physical layer header (PHR) portion and a payload portion, wherein the processor is configured to structure the response message such that the SYNC portion is followed by the SFD

portion, then the cipher portion, then the PHR portion, then the payload portion. In some embodiments, the message further includes a physical layer header (PHR) portion and a payload portion, wherein the processor is configured to structure the response message such that the SYNC portion is followed by the SFD portion then the PHR portion, then the payload portion, then the cipher portion. In some embodiments, the processor is configured to generate two or more cipher portions, wherein the processor is configured to structure the response message such that the SYNC portion is followed by the SFD portion, then the two or more cipher portions, then the PHR portion, then the payload portion. In some embodiments, the processor is configured to generate two or more cipher portions, wherein the processor is configured to structure the response message such that the SYNC portion is followed by the SFD portion, then the PHR portion, then the payload portion, then the two or more cipher portions.

[0007] In some embodiments, a method includes: receiving a message at a transceiver from a second device; generating a response message includes a SYNC portion, a start frame delimiter (SFD) portion, and a cipher portion, wherein: the SYNC portion includes a sequence of symbols forming a preamble; and the cipher portion includes a ciphered sequence of pseudo-randomized pulses; transmitting the response message. In some embodiments, the method further includes generating the cipher portion using a pseudo-random number generator. In some embodiments, the message further includes a physical layer header (PHR) portion and a payload portion, wherein the processor is configured to structure the response message such that the SYNC portion is followed by the SFD portion, then the cipher portion, then the PHR portion, then the payload portion. In some embodiments, the message further includes a physical layer header (PHR) portion and a payload portion, wherein the processor is configured to structure the response message such that the SYNC portion is followed by the SFD portion then the PHR portion, then the payload portion, then the cipher portion. In some embodiments, the method further includes generating two or more cipher portions and structuring the response message such that the SYNC portion is followed by the SFD portion, then the two or more cipher portions, then the PHR portion, then the payload portion. In some embodiments, the method further includes generating two or more cipher portions and structuring the response message such that the SYNC portion is followed by the SFD portion, then the PHR portion, then the payload portion, then the two or more cipher portions.

[0008] In some embodiments, a non-transitory computer readable medium that stores computer executable instructions, wherein, in response to executing the computer executable instructions, the processor is configured to: receive a message at a transceiver from a second device; generate a response message includes a SYNC portion, a start frame delimiter (SFD) portion, and a cipher portion, wherein: the SYNC portion includes a sequence of symbols forming a preamble; and the cipher portion includes a ciphered sequence of pseudo-randomized pulses; transmitting the response message. In some embodiments, the processor is configured to generate the cipher portion using a pseudo-random number generator. In some embodiments, the message further includes a physical layer header (PHR) portion and a payload portion, wherein the processor is configured to structure the response message such that the SYNC portion is followed by the SFD portion, then the cipher portion, then the PHR portion, then the payload portion. In some embodiments, the message further includes a physical layer header (PHR) portion and a payload portion, wherein the processor is configured to structure the response message such that the SYNC portion is followed by the SFD portion then the PHR portion, then the payload portion, then the cipher portion. In some embodiments, the processor is configured to generate two or more cipher portions, wherein the processor is configured to structure the response message such that the SYNC portion is followed by the SFD portion, then the two or more cipher portions, then the PHR portion, then the payload portion. In some embodiments, the processor is configured to generate two or more cipher portions, wherein the processor is configured to structure the response message such that the SYNC portion is followed by the SFD portion, then the PHR portion, then the payload portion, then the two or more cipher portions.

[0009] Those skilled in the art will appreciate the scope of the present disclosure and realize additional aspects thereof after reading the following detailed description of the preferred embodiments in association with the accompanying drawing figures.

Description

BRIEF DESCRIPTION OF THE DRAWING FIGURES

[0010] Preferred features of the present invention will now be described, purely by way of example, with reference to the accompanying drawings, in which:

[0011] FIG. 1 illustrated on-air frames of a UWB (Ultra Wide Band) two-way ranging protocol;

[0012] FIG. 2 illustrates on-air frames of a secure UWB two-way ranging protocol using DecaCypher;

[0013] FIG. 3A illustrates communication with a node having multiple transceivers;

[0014] FIG. 3B illustrates an automatic parking scenario utilizing described communication techniques;

[0015] FIG. 4 illustrates two-way ranging to multiple anchors (cams);

[0016] FIG. 5 illustrates use of quasi-simultaneous frames in accordance with one disclosed communication scheme;

[0017] FIG. 6 illustrates typical carrier loop and accumulators for $\text{CFO} = -5$ ppm;

[0018] FIG. 7 illustrates simulation results for four quasi-simultaneous frames;

[0019] FIG. 8 shows a time domain plot of four quasi-simultaneous frames;

[0020] FIG. 9 illustrates an alternative communication scheme using quasi-simultaneous frames with simultaneous DecaCyphers;

[0021] FIG. 10 illustrates simulation results for four quasi-simultaneous frames with simultaneous DecaCyphers;

[0022] FIG. 11 shows CFO trimming measurements;

[0023] FIG. 12 shows a protocol frame as used, e.g., in a DW1100 implementation;

[0024] FIG. 13 illustrates a variation of the communication scheme with simultaneous DecaCypher sequences as implemented, e.g., using DW1100 transceivers;

[0025] FIG. 14 illustrates four quasi-simultaneous frames, separate and superimposed;

[0026] FIG. 15 shows simulation results of four quasi-simultaneous frames with simultaneous DecaCypher in a DW1100 implementation;

[0027] FIG. 16 illustrates a performance comparison for a two-path channel;

[0028] FIG. 17 illustrates localization in attenuated NLOS conditions;

[0029] FIG. 18 illustrates localization in LOS conditions;

[0030] FIG. 19 illustrates a 4-anchor walk-around test;

[0031] FIG. 20 illustrates distance measurements for the 4-anchor walk-around localization test;

[0032] FIG. 21 illustrates a PRF64 frame structure example (216 ps duration);

[0033] FIG. 22 illustrates a PRF64 frame structure for use in localization with optional frame portions (177 ps frame duration);

[0034] FIG. 23 illustrates a PRF64 frame structure with two valid Ipatov preambles and payloads (175 ps frame duration);

[0035] FIG. 24 illustrates PRF64 packets (with 158 ps frame duration);

[0036] FIG. 25 illustrates a PRF64 scheme with identical data transmitted (163 ps frame duration);

[0037] FIG. 26 illustrates parallel simulranging without data payload (123 ps frame duration);

[0038] FIG. 27 illustrates a basic PRF16 variant;

[0039] FIG. 28 illustrates a PRF16 variant with smaller payload differences and longer Ipatov sequences;

[0040] FIG. 29 illustrates a 2-anchor variant of the communication scheme (158 ps frame

duration);

[0041] FIG. **30** shows an ultra-short 2-anchor variant of the scheme, with (optionally) muted SLAVE CAM_B preambles;

[0042] FIG. **31** illustrates an example scenario of transmission/reception between a vehicle and keyfob;

[0043] FIG. **32** illustrated estimated CIR in the example scenario;

[0044] FIG. **33** illustrates on-air frames of keyfob and anchor;

[0045] FIG. **34** shows a plot of path amplitude vs CFO in a 6.5 GHz channel;

[0046] FIG. **35** shows a zoomed-in plot of path amplitude vs CFO in the same channel.

[0047] FIG. **36** shows a plot of path amplitude vs CFO in a 4 GHz channel; and

[0048] FIG. **37** shows a plot of path amplitude vs CFO in an 8 GHz channel; and

[0049] FIGS. **38** to **41** show additional alternative frame structures.

DETAILED DESCRIPTION

[0050] The IEEE standard 802.15.4-2011 describes an ultra-wideband physical layer (UWB PHY) with the frame structure and two way ranging ability shown in FIG. **1**. The solid outlines show transmitted frames. The dotted outlines show received frames. The following abbreviations are used in FIG. **1** and subsequent diagrams to denote certain portions of the frames: [0051] SFD—Start Frame Delimiter [0052] PHR—PHY (Physical layer) Header

[0053] The frame includes a training sequence used for channel sounding and frame detection, in the form of an Ipatov sequence. This is a sequence of symbols forming a frame preamble, based on a predefined code sequence having certain properties that make it useful for channel sounding purposes (including perfect periodic autocorrelation).

[0054] If Device A knows the sending time of a frame, T_{sa} , and detects the receiving time of the response, T_{ra} , and knows the response delay, t_d , then it can calculate the propagation time, t_p as:

$$[00001] \quad t_p = \frac{T_{ra} - T_{sa} - t_d}{2} \quad (1)$$

[0055] It can then multiply this by the speed of light to find the distance between Device A and Device B.

[0056] Because of multipath propagation, the receiver in each device will receive a signal consisting of multiple delayed, superimposed copies of the frames. The devices use the Ipatov preamble sequence as a training sequence for channel sounding to estimate the impulse response of the channel and therefore calculate the arrival times of each of these copies. They use the arrival time of the first copy as the true T_{ra} .

[0057] The scheme as described in 802.15.4 is not secure because the Ipatov preamble sequence is a periodic sequence that repeats a pre-defined code. This can be spoofed by an attacker to make it appear to have arrived earlier than it actually arrived. For this reason, patent publication WO 2017/134310, the entire contents of which are incorporated herein by reference, introduced a secure scheme where the channel sounding sequence (also referred to herein as a training sequence) can be any random binary sequence that could, for example, be generated by a cryptographically secure random number generator (CSPRNG). In this case both the receiver and transmitter have access to the necessary information (including the generation algorithm and any seed and/or key information) to generate the cryptographic training sequence (at the transmitter), and to regenerate it (at the receiver) to detect the sequence on reception. This makes the channel sounding sequence secure from relay attacks. We shall refer to that sequence as the “DecaCypher.” Note that the DecaCypher sequence is also referred to herein as the “cypher preamble” or simply “cypher” for short. The use of the term “preamble” in this context is not intended to imply that the DecaCypher sequence is located at or near the beginning of the frame, but rather indicates its function as a training/channel sounding sequence; in the presented examples the DecaCypher is typically located at the end or in the middle of the frame (with the Ipatov preamble at the start of the frame).

[0058] One of the ways the DecaCypher is used in the above patent publication is shown in FIG. **2**

(illustrating on air frames of a secure UWB two-way ranging protocol using DecaCypher).

[0059] The following described embodiments use transceivers that implement these frame formats.

[0060] Note that the term DecaCypher as used herein may refer to a secure training sequence as used in any of the embodiments described in WO 2017/134310, but other types of training/pilot sequences may alternatively be substituted. Preferably, these are cryptographically generated/secured training sequences, but in other embodiments (e.g. where security is of less concern) any other type of training sequence may be used as the DecaCypher element of the frame.

[0061] Unless required otherwise by context, the terms “frame” and “packet” are used herein interchangeably to refer to a structured data unit transmitted via the communication medium (in present examples wirelessly).

[0062] Embodiments of the invention employ variations of the above frame structures for communication between nodes, where at least one of those nodes includes multiple separate transmitters/transceivers. An example of such an arrangement is shown in FIG. 3A, which includes a first node **302** and a second node **304**. The second node includes four separate transceivers **306**, **308**, **310** and **312**, which are also referred to herein as “anchors”, each associated with a respective antenna for transmission/reception of signals. Node **302** in this example includes a single transceiver **316** (though in other examples both nodes could include multiple transceivers). Communication between the nodes can occur via any of the anchors of node **304**.

[0063] Described embodiments may use DW1000 or DW1100 transceiver integrated circuit (IC) modules (available from Decawave of Dublin, Ireland) in the transceivers in node **302** and/or node **304**. However, any suitable transceiver technologies may be employed. In one embodiment, node **304** may include a controller **314** (e.g., a microprocessor) for receiving data from the multiple transceivers and for controlling and coordinating transmission of frames from the respective transceivers, using the various techniques, frame structures and frame timings described herein. However, in preferred embodiments the various communication schemes described herein operate without central coordination of the anchors. In particular, the anchors each receive messages from node **302** independently and generate response frames autonomously. However, since the anchors' responses are generated with predetermined timings relative to reception of messages at the anchors, the respective reply frames are still generated in a coordinated fashion and substantially simultaneously, except for minor differences due mainly to differences in signal propagation times between node **302** and the different anchors of node **304**.

[0064] An example real-world application is illustrated in FIG. 3B. Here, node **302** is a wireless keyfob for a car or other vehicle, and node **304** is the vehicle, which includes the transceivers or anchors **306-312** (e.g., positioned at respective front and rear corners of the vehicle). In this embodiment, the described communication schemes are used to implement an automatic parking function of the vehicle.

[0065] In this scenario, a car is being parked automatically with the driver nearby to make sure nothing goes wrong. Regulations state that the driver shall be no more than 6 m away from the car. There is a transceiver on each corner of the bumpers of the car ready to respond to a signal from the keyfob. At any particular time, the driver may have an unobstructed view of only one of the antennas and the best antenna to use may change constantly as the car maneuvers itself. For safety reasons, if the car stops receiving an “All Clear” signal from the keyfob, the car must stop the automatic parking procedure within a short time, e.g. of the order of **100 ms**.

[0066] One way to address this is for the keyfob to transmit a frame every 100 ms. Each of the four transceivers/anchors in the car, which we shall refer to as “Cams,” is programmed to respond to the keyfob after a different pre-programmed response time, e.g., after 20 ms, 40 ms, 60 ms and 80 ms. Since the keyfob has a relatively clear view of at least one antenna, and since it is pre-programmed to be ready and listening for each of these at the appropriate time, it will usually receive at least one of the frames. This communication scheme is illustrated in FIG. 4.

[0067] We can re-use equation (1) to find the propagation delay to each Cam, e.g., for Cam C:

$$[00002] \quad tp_c = \frac{Trk_c - Tsk - td_c}{2} \quad (2)$$

[0068] There are a few problems with this. One is that the keyfob usually has a small capacity battery and it takes a lot of power to send one frame and then receive four frames. In addition to battery capacity, it will often require large capacitors to avoid drawing too much instantaneous current from the battery. The more frames that are received, the larger these capacitors must be. Ranging With Simultaneous Frames

[0069] Embodiments of the invention provide an alternative solution, using a scheme referred to herein as “simulranging”. While for clarity and ease of understanding the scheme will continue to be described in the context of a keyfob communicating with a vehicle—e.g. in the automatic parking scenario or in a secure door locking/unlocking application—it should be noted that the scheme is not limited to this application but can be applied in any appropriate context involving communication between nodes or devices, at least one of which includes multiple transceivers.

[0070] In this scheme, all four cams send their frames quasi-simultaneously, but only the one is received by the keyfob. Each transceiver in the car is programmed to respond to an initial message from the keyfob after a slightly different time delay, for example, 100 ps, 101.25 ps, 102.5 ps and 103.75 ps respectively, from reception of the message from the keyfob at the respective transceiver. This means that they are all responding almost simultaneously, but with an important 1.25 ps spacing. The receiver in the keyfob is programmed to be ready to receive a frame after 100 ps and it will “see” some of the frames, maybe only one but maybe all of them. The 1.25 ps gap means however that it only actually decodes the frame of one of the responses and only estimates the channel impulse response of one of the frames, because the symbol boundaries of the respective Decacypher training sequences (which have symbol durations of approximately 1 ps) are not aligned. More generally, the respective offsets may be any non-integer multiples of the Decacypher symbol duration so as to prevent symbol alignment of the Decacypher sequences.

[0071] Thus, the specific offsets are given purely by way of example (elsewhere herein, offsets of e.g., 128 ns are suggested). It is preferable that the offsets are larger than the expected delay spread (e.g., for multipath delays), so that (for example) multipath from one transmitter would not overlap its energy onto the accumulator area of another transmitter.

[0072] The frames on the air, as described, will appear as shown in FIG. 5.

[0073] This is what the receiver (e.g., the keyfob) does as it receives a frame: [0074] Starts up in Ipatov preamble detection mode. [0075] In preamble detection mode, it continually assumes that a signal is present and constructs CIR (Channel Impulse Response) estimates for whatever it receives. [0076] When it decides that the CIR estimate is coming from an actual valid signal it switches to acquisition mode. [0077] In acquisition mode it uses an accumulator to sum many noisy channel estimates to construct a 1 ps long channel impulse response (CIR) and starts the carrier recovery algorithm. [0078] The carrier recovery algorithm only gathers information from a 64 ns section of the full CIR that contains the most energy. [0079] In acquisition mode it is also searching for the SFD signaling the end of the preamble. Again, just gathering information from the 64 ns window. [0080] When the SFD is found the receiver prepares to receive the payload. The Ipatov CIR estimate is complete. [0081] The payload is demodulated, again, only using information from the same 64 ns section of the CIR.

[0082] When the payload demodulation is finished, it constructs a second, secure, CIR estimate using the Decacypher.

[0083] FIG. 6 shows the accumulators and the carrier loop for a typical receive frame which has a −5 ppm (parts-per-million) carrier frequency offset (CFO) and where there is a weak path followed by a peak path which is 10 dBs stronger and delayed by 15 ns.

[0084] The top subplot is the carrier recovery loop homing in on the −5 ppm offset. The second subplot shows the accumulator formed from preamble detection. The receiver picks the strongest signal here and adjusts its delay so that this signal appears at approximately tap index 728. It then

focusses its carrier and timing recovery on this signal during reception of the Ipatov preamble and accumulates its Ipatov based channel estimate. The third subplot shows the channel estimate built up from the Ipatov preamble. The fourth subplot shows the channel estimate built up from the DecaCypher.

[0085] A simulation of a receiver with all four of the frames shown in FIG. 5, arriving simultaneously, was simulated in Matlab. Four frames with different carrier frequency offsets (CFOs) were transmitted almost simultaneously. The receive power for 3 of the frames was -90 dBm, while the fourth frame receive power was -88.5 dBm. The channel for each signal was a two-path channel where the first path is 10 dB weaker than the main path: [0086] Frame 1) Sent after a delay of $40\ \mu\text{s}$. $\text{Cfo} = -5$ ppm. Rx level = -90 dBm. [0087] Frame 2) Sent after a delay of $41.25\ \mu\text{s}$. $\text{Cfo} = -4$ ppm. Rx level = -88.5 dBm. [0088] Frame 3) Sent after a delay of $42.50\ \mu\text{s}$. $\text{Cfo} = -3$ ppm. Rx level = -90 dBm. [0089] Frame 4) Sent after a delay of $43.75\ \mu\text{s}$. $\text{Cfo} = +5$ ppm. Rx level = -90 dBm.

[0090] FIG. 7 shows the receiver response to the (quasi-) simultaneous frames according to the simulation.

[0091] The top subplot is the carrier recovery loop homing in on the -4 ppm offset of the strongest signal. The second subplot shows the accumulator formed from preamble detection. The receiver picks the strongest of the four signals seen here and adjusts its delay so this strongest signal appears at approximately tap index **728**. This will generally be the nearest line of sight (LOS) corner of the car. It then focusses its carrier and timing recovery on this signal during reception of the Ipatov preamble and accumulates its Ipatov based channel estimate. The third subplot shows the channel estimate built up from the Ipatov preamble. Three of the four frames produce strong paths in this estimate. The signal due to the fourth frame is not present because its CFO is 9 ppm different than the signal being followed. The fourth subplot shows the channel estimate built up from the Cyphered preamble. Only one frame produces any paths in this estimate. The signal due to the second frame is the only one whose delay matches the delay the receiver has homed in on. The other three frames are more than 1 ps away. The Decacypher uses groups of symbols of length 1 ps, so symbols which are more than 1 ps away from another symbol have zero correlation with it.

Security Enhancements

[0092] The simulranging scheme as outlined above is not secure, even though the Decacypher is being employed. The four frames all have the same Decacypher sequence, but they are sent with a delay between them. An attacker could listen to the first sequence and he would then have 3.75 ps to use a record and replay attack on the fourth frame sending the same sequence slightly earlier than the one he knows will be transmitted hence reducing the true time of flight (TOF). This may not matter in an Auto Parking application, but it may matter in other applications, e.g., in a passive entry system.

[0093] The scheme can be modified to be secure as shown in FIG. 8 and FIG. 9 and described below.

[0094] FIG. 8 shows a time domain plot of four quasi-simultaneous frames.

[0095] FIG. 9 illustrates quasi-simultaneous frames with simultaneous DecaCyphers. Here, each frame is sent as before with a time offset of 1.25 ps (relative to a preceding frame sent by another transceiver), but the DecaCyphers are all sent aligned, i.e. in each frame, a gap is inserted between the payload and the Decacypher so that the sum of that inserted gap and the time offset at the start is equal to some constant so that each Decacypher is transmitted with the same delay after reception of the keyfob message by the transceiver—and thus each Decacypher is transmitted in effect simultaneously from each of the four transceivers (except for minor variations due to different propagation times between the keyfob and the different transceivers).

[0096] Of course, the receiving IC in the keyfob only “sees” one of the four frames and it cannot tell which one that is, so it does not know the gap between the payload and the Decacypher. For this reason, it signals the length of the gap before the Decacypher arrives. This gap length can be

encoded in the PHY header (PHR) or in the payload. The payload is typically encrypted, so it may take some time to decode, for this reason, the PHR may typically be a more suitable place to encode it. The PHR is not encrypted, so there is the potential to attack it and perhaps fool the receiver into thinking it came earlier than it actually did. To combat this concern, the transmitter can also encode the same bits in the payload (possibly encrypted) for later verification.

[0097] This is now secure and saves a large amount of energy in the keyfob. It only needs to do one frame reception but receives the frame from the best positioned transmitter.

[0098] In this revised secure scheme, it may happen that all the signals at preamble detect time are of a similar strength. The receiver might then pick out the furthest away signal to train its carrier and timing recovery and compose its channel estimate. The cyphered signal would then arrive, and it could happen that a closer transceiver has a very similar CFO. The cypher pulses from this transceiver will appear ahead of the pulses of the farther transceiver and seemingly corrupt the channel estimate. In fact, this is not a corruption, this is just indicating that there is actually a closer transceiver than the one that was originally locked onto.

[0099] FIG. 10 shows the corresponding algorithm outputs for this case (simulation of 4 quasi simultaneous frames with simultaneous DecaCypher).

Adjusting Clocks in the Anchor to a Target Offset

[0100] When there are different clock offsets between the keyfob and the transceivers, the precise values of the various delays may change by a small fraction, depending on which device is counting the delays. These delay variations can add up and cause perceived arrival times to move around. This can cause complications. Embodiments of the invention address this utilizing a facility provided in some modern ICs for making small adjustments to the frequency of the source clock by, for example, making small adjustments to the values of the crystal trimming capacitors. They can achieve a target frequency with an error of less than 1 ppm. In many receivers, the CFO can be measured. This allows the IC, after receiving a frame at a particular CFO and having measured that CFO, to trim its source clocks to be a desired value. This would take effect before they transmit their response to that frame. For example, to make sure all the receivers in the car start their silence transmission at exactly the same time, they could all set their CFOs relative to the keyfob, to the same value. In many receivers a small negative offset provides other benefits, so they could all aim for a CFO of say, -2 ppm. For other reasons, it might be desirable to have different clock offsets for each receiver, and they could each have different targets, e.g. -20 ppm, -15 ppm, +10 ppm and +20 ppm.

[0101] An important aspect of the crystal trimming procedure is that it allows all anchors to get synchronized to the same CFO. Therefore, even if keyfob locks on to one anchor, paths from other anchors will also be easily seen in keyfob accumulators. This is explained in more detail below. Alternatively, if the CFOs are not tuned to a common CFO, then it is likely that only the anchor to which the receiver synced during acquisition of the Ipatov preamble will be seen.

[0102] If the initial CFO is large, the first trimming may not result in perfect convergence to the target CFO. This is due to a certain unpredictability of the crystal trimming function and its variability from device to device. In embodiments, crystal trimming is therefore done in two steps. The first step involves an initial short message from the keyfob to the anchors consisting only of short Ipatov preamble (and possibly with SFD to confirm preamble validity). There could be no cypher or data payload. Due to that, Ipatov preamble length could be reduced to a minimum, for example 16, 24 or 32 symbols. Crystal trimming is performed initially based on this preliminary message, and subsequently repeated based on the full data frame as previously described to improve convergence to the target CFO.

Compensating for 8 ns TX Transmission Grid

[0103] The digital part of the DW1100 IC is clocked with a 125 MHz clock. This means that it can only start transmission at predefined 8 ns time grid clock edges. That could mean that depending on the packet, transmission can be delayed by 0 to 8 ns from the ideal transmit time and this delay

varies from packet to packet. Normally, the data payload would be used by the transmitter to inform the receiver what delay was required, and the receiver would subtract it. With simulcasting and multiple packets, it is impossible to decode all 4 payloads and compensate for all 4 packets. Without this compensation the first path uncertainty can be up to ± 4 ns. This is discussed further in the “Example Scenario” section below.

[0104] In an embodiment, the 8 ns transmit time uncertainty is removed in the transmitter. Once the offset of the grid time from the ideal time is calculated, the transmitter alters its crystal clock rate and speeds it up or slows it down by a certain number of parts per million (ppm) for a certain time, depending on the compensation required. For example, speeding up the crystal by 30 ppms for 100 ps moves the grid point edge earlier by 3 ns.

[0105] The correction may be done relative to the later 8 ns grid edge by speeding up the clock or to the earlier 8 ns grid edge by slowing down the clock.

[0106] Accurate compensation can be controlled by varying both the crystal trim (range available is typically of the order of ± 40 ppm) and the de-tuning period (which typically could be between 100 and 300 ps).

Transmit Power Reduction

[0107] Embodiments may implement various approaches to controlling the transmit power. TX (transmission) power can be changed per anchor and also individually within each transmitted packet (for example certain parts, like the payload, can be transmitted at a lower power level than the preambles). Reducing transmit power for a given portion of the frame or disabling transmission for a portion altogether (or equivalently reducing transmit power to zero for the portion) is also referred to herein as “muting” the portion.

[0108] If the whole car is treated as one transmitting device, then the transmit power of each individual transmitter (anchor) can be reduced to avoid exceeding aggregate power limitations. In one approach, if another protocol is available to pre-select anchors to transmit, each individual anchor may have its TX power set dynamically within a total TX power budget, depending for example on expected signal strength, number of anchors transmitting, etc.

[0109] In one variation, the keyfob initiates the exchange and anchors respond, and the anchors adjust their TX PWR (transmit power) to prevent ‘shadowing’ of other weaker anchors by its very strong signal. An anchor which sees a very strong 1.sup.st path from the keyfob can significantly reduce its transmit power knowing that the keyfob will have no problems seeing its 1.sup.st path, even after power reduction. Power reduction by the strongest anchors makes it easier for the keyfob to see weaker 1.sup.st paths from other anchors. The algorithm used to reduce the anchor's TX power can take into account multiple factors, for example, Receive level (RSL) of the 1.sup.st path, total RSL including multipath received from the keyfob, knowledge of transmit power applied at the keyfob side, the anchor's own minimum/maximum/default TX PWR settings, the number of anchors potentially transmitting, the lengths of the packets, total TX PWR budget, minimum receiver RSL required to detect weak path, etc.

[0110] Some schemes presented in this document suggest reducing or muting TX power in certain parts of a frame (for example during the Ipatov preamble or data payload) and boosting power during the cyphered training sequence (DecaCypher). In the case of an attenuated (weak) 1.sup.st path, this approach can maximize chances of detecting this 1.sup.st path.

Stopping the Receiver/Transmitter Before the End of the Packet

[0111] This technique allows for receiver and/or transmitter power saving by shutting down the receiver or transmitter RF chain early. For example, in cases where a data payload is not needed but a PHR and data is still transmitted after the cyphered preamble (DecaCypher), the receiver/transmitter can be shut down immediately after the last cyphered symbol.

[0112] In order to stop the RF chain immediately after the cypher, the following pseudo-code is executed: [0113] 1. Trigger an interrupt after SFD detection (or SFD transmission)—this takes around 20-30 ps. [0114] 2. In the interrupt, issue an SPI command to turn off the receiver or the

transmitter—this takes around 10 ps. The cypher takes 64 ps and we don't want to cut it off in the middle, so a delay is used. [0115] 3. Setup a timer after the SFD detection interrupt and then send a command to stop receiving/transmitting at the estimated time.

Shortening the Ipatov Preamble Length

[0116] The channel estimate is used to set the coefficients of the channel matched filter (CMF). This filter is much shorter than the complete channel estimate, e.g., in the DW1000 it is 64 samples long, whereas the channel estimate is about 1000 samples long. The carrier recovery, SFD detection and data demodulation blocks are fed energy from the output of CMF, so that a restricted portion of the complete channel estimate taps are used for these functions.

[0117] The Ipatov sequence can be shorter than usual if a good initial CFO estimate is available or for other reasons. Circumstances allowing this could include: [0118] Where previous messages were received from the same device which have already meant that the receiver knows that CFO target will be close to zero. [0119] Where previous messages were received from the same device which have already meant that the receiver knows that CFO target will be close to zero. [0120] If all anchors transmit at the same time, within the same receiver CMF window, which determines samples used for carrier recovery locking (that is, without 64 or 128 ns timing offsets). [0121] Weak path detection in the resulting Ipatov accumulator is not a concern because only the cypher accumulator is used for first path detection. [0122] There is no data demodulation so no need for an accurate CMF for data demodulation (CFO and SFD detection don't require the CMF to be as accurate as data demodulation does).

Implementing Secure Simulranging With the DW1100

[0123] Note that the following sections provide additional detail on implementing the described techniques using a transceiver based on the Decawave DW1100 transceiver IC. However, the described approaches may be adapted to any other suitable transceiver technology.

[0124] The Decawave IC (DW1100) utilized in these embodiments does not allow the secure simulranging scheme to be implemented as described above and illustrated in FIG. 9 (using a variable transmission gap prior to the DecaCypher field). However, instead, frames can be configured as shown in FIG. 12.

[0125] In this scheme, in order to maintain a 128 ns timing difference between Ipatov pulses (to avoid multipaths adding within the same CMF window), payload sizes are varied so as to differ by 16 octets. This is because each payload symbol is 8 ns longer than an Ipatov symbol, thus introducing 168 ns=128 ns timing shift. While payload sizes differ by 16 bytes, the smallest payload can be only 1 octet in size (the next ones would then be 17, 33, 49 bytes respectively).

[0126] Using this approach, the four Anchors/CAMs in the car are configured to respond as shown in FIG. 13, where Ipatov preamble and payload lengths are varied to balance out the varying delay at the start of each frame.

[0127] The Ipatov preamble and SFD symbols, (shorter “type S” symbols), are all –1016 ns long whereas the PHR symbols, Payload octets and Decacypher symbols, (longer, “type L” symbols or bytes), are all –1024 ns long. This means that if a payload octet is removed from the frame, and at the same time one Ipatov symbol is added, the total frame length decreases by ~8 ns. In this way, we can shorten the portion of the frame that comes before the DecaCypher to offset the amount of silence that comes before the individual responses so that all four of the DecaCyphers are coincident in time and therefore none of them can be predicted by listening to one of the other ones. The receiver knows when the Decacypher will arrive because the payload length is encoded in the PHR. As an aside, the PHR is not protected by an encryption scheme, but when security is required, the payload will be encrypted. Some hacking techniques can make use of this, so, if security is required, the encrypted payload can also contain a confirmation of the payload length.

[0128] In some cases, there can be a slight complication in setting the payload sizes in this way. The Reed-Solomon code adds an error correction block of 6 bytes for every 330 bits in the payload. This means that if any of the payload sizes goes above 41 bytes, a second check block will be

added to it. This requires the other payload blocks to be padded with 6 dummy octets. Of course, the frame durations could also be equalized by increasing the initial silence time, and/or adding extra Ipatov preamble symbols.

Why Do These Simultaneous Frames Not Interfere With Each Other?

[0129] To understand why the above scheme works, it helps to look at an example simulation. FIG. **14** shows an example four frames all on the air at the same time. Each frame has a different silence inserted at the start. The silence lengths are 192 ns, 256 ns, 128 ns, 0 ns, respectively. Then each frame has a different Ipatov preamble length. The lengths are 72, 64, 56 and 48 symbols, respectively. The payload lengths are 12, 20, 28 and 36 octets, respectively. This ensures that each frame's Decacypher is sent at the same time which is important to maintain security. FIG. **14** shows the four quasi-simultaneous frames separately in the first four plots and superimposed in the final plot.

[0130] FIG. **15** shows the results of a simulation of the DW1100 IC in this situation. In this case, each frame was simulated as being on the air with carrier frequency offsets (CFOs) of -2 ppm, -3 ppm, -4 ppm, and -5 ppm, respectively. The frame amplitude was scaled so that the -4 ppm frame was at -89 dBm and the other three frames were at -91 dBm.

[0131] The preamble detection algorithm builds a short snapshot of an estimated channel response. This can be seen in the 2.sup.nd subplot in FIG. **15**. The receiver looks for the strongest energy window and adjusts the receiver delay so that it positions this energy and its channel matched filter (CMF) at the 64 ns window between taps **728** and **792** of the accumulator. This portion of the estimated channel response is used to set the coefficients of the channel matched filter. In this particular simulation, the strongest signal seen during preamble detection was the second one to arrive. Once preamble has been detected, the carrier recovery, the timing recovery and the SFD detection use only the energy in the CMF portion of the channel estimate to operate. This causes the estimated carrier offset being tracked in the carrier loop, to converge to -4 ppm as can be seen in the 1.sup.St subplot of FIG. **15**. The accumulator then builds up its channel estimate until the SFD is detected. This can be seen in the 3.sup.Rd subplot of FIG. **15**. The second arriving signal appears the strongest for two reasons. Firstly, because the signal is actually 2 dB stronger and secondly, because the other signals' CFOs are not being corrected which causes these other signals to partially cancel.

[0132] The SFD of the second arriving signal is detected. Note that any SFDs which arrive earlier will not be seen, because they do not arrive within the window where the CMF is positioned. The span of the CMF is only 64 ns and the four signals are all delayed by at least 64 ns from each other.

[0133] Next, the receiver starts decoding the data to receive the data bits in the PHR and then the payload. It uses the CMF, which only gathers energy from the window where the wanted signal is, so that the energy of the data bits of the other frames is not raked in by the CMF. Also, each bit of the PHR and the following data uses a different spreading code in the transmitter and despreading code in the receiver, which further isolates the true data from the unwanted other signals.

[0134] Next the receiver gets all four DecaCypher signals from all of the frames at the same time separated by the individual propagation delays. The receiver knows the polarity of each pulse in the sequence of pulses that make up each symbol in the DecaCypher, so it can correlate with the expected pulses and get an estimate of the channel impulse response. Since there are up to four copies of the DecaCypher sequence arriving at the receiver at a similar time, the impulse response estimate will be estimates of all four channels overlaid. In this way, when it uses this to calculate the distance to the car, the first arriving signal it will see, will be the first large enough path of all of these DecaCyphers that arrives.

[0135] FIG. **16** illustrates a performance comparison for single packet vs. four simultaneous packets (2 path channel).

[0136] Note that in the above examples, particular delays, silences, Ipatov lengths and payload sizes are used. There are many other delays that can be used to balance, or approximately balance,

the silence with the frame lengths to achieve the same effect. There are many other CMF lengths than 64 ns that could be used. Symbol lengths can vary from 1 ps, without changing the underlying ideas. Also, the above examples are for the 6.8 Mbps mode described in IEEE802.15.4-2011. There are other bit-rates that could be used, requiring different numbers to balance the delays, also there are other wireless implementations, including IEEE802.15.4f, IEEE802.15.6, IEEE802.15.8 and many others, which would benefit from the techniques described here.

Simulranging-Based Localization

[0137] Localization via triangulation methods is also possible with simulranging packets. To triangulate, distances from two or more anchors to the keyfob are obtained. Since all cypher transmissions are aligned, it is generally not possible to separate various anchor paths in the cyphered accumulator. However, it is possible to calculate different distances based on sections of the Ipatov accumulator.

[0138] To do this, pulse sequences from anchors are shifted, for example by 128 ns, so that they occupy different segments in the Ipatov accumulator.

[0139] An example of using this shifting technique in a challenging NLOS (non-line-of-sight), scenario (e.g., keyfob in back pocket) with 4 anchors hidden behind a body is shown in the accumulator plot in FIG. 17. The first 1016 accumulator samples are the accumulator from the Ipatov preamble. The next 512 samples are the accumulator from the cyphered preamble (DecaCypher). Hypothetical Ipatov sample indexes corresponding to 0-meter distance are computed for each anchor (vertical red lines). The distance from these lines to the first path determines the distance of each anchor—in this example, 2.1 m, 1.3 m, 1.6 m, 1.7 m—which are close to the actual distances. The cypher-based distance was calculated as 1.5 m therefore in the triangulation process, for security reasons, the shortest Ipatov-based distance is increased from 1.3 to 1.5 m.

[0140] In LOS (line-of-sight) conditions, the paths are stronger and sharper, except when one anchor is at extremely close distance and blasting with huge power. In FIG. 18, anchor #3 is at 0.1 m distance which reduces amplitude of other LOS paths at 2.0-2.6 meters.

[0141] Walk-around test results are presented below. Four dots representing 4 anchors were placed in a square approx. 1.9 m from each other as illustrated in FIG. 19. The walking route is shown.

[0142] Determined distances to all 4 anchors (based on individual Ipatov preambles) are presented in FIG. 20, together with cypher distance (i.e. the distance determined based on the simultaneous DecaCypher training sequence).

Simulranging Packet Structure Optimizations

[0143] Compared to the baseline proposal shown above, many optimization approaches are possible: [0144] 1. Reduce Ipatov pulses timing offset. The timing offset may be reduced from 128 ns to 96 ns or even 64 ns. That should still be sufficient to prevent multipath overlapping CMF window of another path. The consequence would be reduced differences between payload sizes. In PRF64 mode, instead of payloads differing by 16 octets (resulting in 128 ns timing offset), they could differ by 8 (64 ns offset). That would help reduce overall packet size. [0145] 2. Reduce data packet size. If the minimum data packet block can be, for example, 4 bytes instead of 16, then ALL payload sizes could be reduced by 12 bytes (12 ps). It is also possible to send PHR without any data octets (0 size) if data payload is not needed; in that case, payload sizes could be 0, 16, 32, 48 bytes for four anchors. [0146] 3. Reduce TX power/mute. Since the keyfob only needs to lock on to and decode a payload from one anchor, the other anchor Ipatov/payloads can be delayed and muted (or transmitted at lower power). This way, total TX power budget can be optimized and the power of selected sequences can be increased. In the case of packet structures, where only certain anchors send a valid payload (MASTER anchors), one of the remaining anchors (SLAVE), can operate in an alternative cypher mode where the payload is transmitted after the cypher (thus reducing the overall packet length). [0147] 4. Pre-selection. If another physical communication layer (like LF), or based on previous UWB packet transmissions, the system can do anchor pre-selection of, for

example, the 2 best anchors with the strongest signal, 2-anchor variants would be possible, significantly reducing packet length increase compared to 4-anchor variants. Alternatively, the 2 best pre-selected anchors can send Ipatov preambles with DATA, while the remaining can transmit only a cyphered preamble (with a muted Ipatov preamble, SFD, PHR/DATA)—See FIG. 22. [0148] 5. Add silence gap to avoid locking on to shortest Ipatov preamble. In order to improve the Ipatov accumulator quality (for example, for localization purposes), it would be best to allow it to accumulate for longer than the shortest Ipatov length. To prevent locking in on the shortest Ipatov (and stopping accumulation after 1.sup.St SFD), the shortest accumulators can be preceded by at least 8 symbols-long silence. This will cause the keyfob to lock on to the earlier arriving Ipatov preambles. To improve the quality of the channel estimate from the shorter Ipatov anchors, their Ipatov TX powers could be increased at the expense of the data payload (which will not be decoded anyway). See FIG. 21. [0149] 6. Using mixed cypher protocols. Using cypher protocol 1, where the cypher is sent immediately after the SFD, on certain anchors (there will be no 1024 ns symbols before cypher) and certain anchors with payload size 0 (only PHR) which has 21 1024 ns symbols before the cypher. Since the receiver is configured for either mode 1 or mode 2, anchors operating in a different mode should be SLAVE anchors (i.e. the receiver will not lock on to them). See FIG. 22. [0150] 7. Using cypher-gap. In cases of anchors in SLAVE mode (the keyfob will never lock on to them), one can use a configurable transmission gap before the cypher preamble instead of a long payload or long Ipatov. This allows transmission power to be saved and the power budget can be used to boost the power of other sequences. [0151] 8. Early RF shutdown. The RF receiver and/or transmitter can be shut down immediately after receiving/transmitting the cypher, not waiting for PHR/DATA. This will allow for further packet length reduction by at least 21-28 us. See FIG. 26. [0152] 9. Parallel packets without timing offset. With crystals trimmed and all anchors having the same CFO, and with identical data payload (or no data payload at all) it is possible to transmit completely in parallel without any timing offsets. Additional anchors would then look like multipaths. Certain anchor messages can have optionally muted Ipatov/SFD preambles to allow for a TX PWR increase of active sequences. [0153] 10. Shortening the Ipatov preamble. With CFO known to the keyfob and with a strong signal coming from multiple anchors, it should be possible to shorten the Ipatov preamble.

[0154] And several PRF16-based optimizations: [0155] 11. Using PRF16 mode. Ipatov symbol length is 992 ns allowing a reduction in the length difference between the payloads of anchors from 16 to just 4 bytes. That will allow for shortening of packet lengths or extending effective preamble lengths (at the same packet size) for better localization performance. [0156] 12. A mixed mode, using PRF16 Ipatov with PRF64 cypher is also possible. It would optimize payload sizes while providing the best cypher quality. PRF16 Ipatov has better sensitivity than PRF64 Ipatov because all Ipatov sequences in PRF16 have nearly the same length. For example, length-88 PRF16 may perform better than 48-length PRF64—and both would have the same packet size. In the cyphered preamble, symbol count is the same irrespective of PRF and PRF64 performs better.

[0157] There is huge number of legal combinations, which should work and result in aligned cypher preambles. Depending on scenario and requirements, a user can apply the presented ideas selectively, to optimize packet structures. Several scenarios are presented in examples below.

PRF64 Variants Support Localization

[0158] Localization requires that all Ipatov sequences be shifted by a known timing shift (for example 128 ns) apart. This timing shift can be implemented by increasing the number of data symbols (each 1024 ns long) or enabling/disabling the PHR (which also consists of 1024 ns symbols).

[0159] The following examples implement optimization ideas set out in the previous section.

[0160] Referring back to the default structure shown in FIG. 12, the packet length for that arrangement is:

$$[00003]384 + 96 * 1016 + 8 * 1016 + 21 * 1024 + 1024 * 22 + 65536 = 215616\text{ns} = 216\mu\text{s}$$

[0161] One weakness of the default structure is that locking to CAM_A with the shortest Ipatov preamble will stop the accumulation process after 40-48 symbols. That will degrade the quality of Ipatov peaks.

[0162] In a proposed alternative, due to CAM_A/B starting delay, the keyfob will lock to either anchor C or D, but paths from A and B will be present in the Ipatov accumulator. Accumulation will now take 80 or 96 symbols, maximizing anchor C/D energy in the Ipatov accumulator. Since the keyfob will not lock to A or B, their payloads can be transmitted using minimal power. CAM A and B can use their TX power budget to instead transmit their Ipatov preambles at a higher TX power to compensate for their shorter length. While this alternative has the same length as the default one (216 ps), it should have much better Ipatov peak quality for more accurate localization.

[0163] If a data payload is not needed, it is possible to reduce the packet lengths (shortening all payloads by 22 symbols).

[0164] FIG. 21 illustrates a PRF64 frame structure example (216 μ s).

[0165] FIG. 22 presents a shorter alternative, where the packet length from the start till the end of DecaCypher is just:

$$[00004] 424 + 131016 + 881016 + 81016 + 65536 = 176704\text{ns} = 177\mu\text{s}$$

This is achieved by setting up the 1st anchor with PHR/DATA after the cypher and the 2nd just with a PHR. Using only PHR without DATA (21 symbols*1024 ns) shifts Ipatov pulse grid by $21*8\text{ ns}=168\text{ ns}$ which is more than sufficient.

[0166] Placing >8 symbols of silence before Ipatov will result in the keyfob not locking to the anchor. A possible weakness of this scheme is again locking in to shorter Ipatov sequences (CAM_C or CAM_D) and generally shorter accumulation lengths.

RF64 Variants Without Localization but With Anchor-Specific Data Requirement

[0167] In many cases where localization is not required, especially with anchor pre-selection, there is no need to transmit 4 payloads or 4 Ipatov sequences and a reduced number of valid Ipatov/payload messages can be sent. In the example shown in FIG. 23, only CAM C and CAM D transmit valid Ipatov preambles and payloads. The keyfob will lock on to one of these, while still receiving the cyphered preamble from all 4 anchors. Not transmitting a valid data payload on some anchors allows for overall multi-packet length reduction. In the example here, the packet length (till the end of the cypher) is:

$$[00005] 0 + 48 * 1016 + 8 * 1016 + 21 * 1024 + 30 * 1024 + 65536 = 174656 = 175\text{ ns}$$

[0168] CAM A and B are mostly muted except for the cypher preamble (DecaCypher), so they could alternatively be programmed in the same way as CAM_C/D but with Ipatov/SFD/PHR/DATA muted.

[0169] If a single anchor is deemed sufficient (to receive the payload from), then CAM C could be setup identically as CAM A B with only CAM D transmitting valid payload and Ipatov. This would shorten the packet even further, as illustrated in FIG. 24:

$$[00006] \text{Length: } 48 * 1016 + 8 * 1016 + 21 * 1024 + 14 * 1024 + 65536 = 158272\text{ns} = 158\text{ ns}$$

Variants Without Localization Support and With Identical Data on All Anchors

[0170] With accurate crystal trimming on all anchors, it would be possible to transmit from all anchors at the same time without timing shifts (64, 128 ns, etc.). Such transmissions would be seen by the keyfob as additional paths in a multipath profile. Because all paths would end up in the same CMF window (64 ns wide), all transmissions need to have identical data and very closely matched CFO. Therefore, if there is no requirement to transmit anchor-specific data (e.g., anchor ID), or if there is no requirement to transmit any data at all, the scheme depicted in FIG. 25 could be used.

[0171] Additional anchors can either be muted during Ipatov/SFD preamble or transmitting. The scheme could be implemented with any number of anchors. Due to potentially better Ipatov signal quality received by the keyfob, and due to controlled CFO (which is known a priori to the keyfob), it could be possible to shorten the Ipatov sequence.

Variants Without Localization Support and Without Data Requirement

[0172] If no data is required, each anchor can be configured with PHR/DATA (0 size) to be transmitted after the DecaCypher field and with its TX PWR muted. The receiver in the keyfob can therefore shut off its RF immediately after the cypher (similarly the transmitter can shut down its RF after the cypher). This technique was discussed above. An example is shown in FIG. 26. With a 48-symbol Ipatov, the total sequence length is 123 ps (excluding the unused PHR), but can be shortened further if a shorter Ipatov preamble can be used (as discussed previously).

PRF16 Ipatov Variants Supporting Localization

[0173] In PRF16, the Ipatov/SFD symbol length is 992 ns, 32 ns shorter than payload symbol length. This makes scheduling of the packets more optimal because payload sizes can now differ by only 4 bytes (4 symbols times 32 ns difference gives 128 ns timing shift between packet pulses during Ipatov).

[0174] FIG. 27 presents an example PRF16 structure. Note that all payloads can be decreased by the same number of bytes (for example to 4, 12, 20, 28 bytes) and the Ipatov preamble can be reduced to, for example, 48 symbols (CAM_A), 56 (CAM_B), 64 (CAM_C), 72 (CAM_D).

[0175] Further optimization can be done reducing payload length differences to 4 bytes. In this scheme, adding 4×992 ns silence to CAM_B and CAM_D allows only two Ipatov lengths to be used. And as in previous example, payloads could be reduced to 4, 8, 12, 16 octets and Ipatov sequences to 48 symbols (CAM_A and CAM_B) and 56 symbols (CAM_C and CAM_D). An example is illustrated in FIG. 28, showing a PRF16 structure with smaller payload differences and longer Ipatov sequences.

Packet-Length Optimized 2-Anchor Variants (With and Without Data)

[0176] 2-anchor variants can be derived from the previously described 4-anchor variants.

[0177] An example (158 ps) with a small payload is shown in FIG. 29. The Ipatov preamble and data payload can be transmitted on one or both anchors. If data is transmitted on both, it should be identical.

[0178] If no payload is needed, it is possible to move the PHR/DATA after the cyphered preamble. This is illustrated in FIG. 30. CAM_B Ipatov/SFD can be either muted or transmitted at lower power, the PHR/DATA is always muted. This provides an ultra-short 123 ps packet (or shorter if Ipatov can be shortened), assuming transmitter/receiver RF is shut down (as explained in a previous section) after the DecaCypher sequence.

[0179] Advantages of these variants may include: [0180] Ultra short packet size for lowest possible power consumption. [0181] Since TX PWR limit is calculated per 1000 ps, we can increase baseline TX PWR $1000/123=8.13$ times, that is by 9.1 dB (assuming 123 ps length). [0182] Optionally SLAVE CAM_B can only transmit Cypher (64 ps). Approx. 50% TX PWR saved by SLAVE can be transferred to boost the overall TX PWR budget.

[0183] If SLAVE CAM_B is not transmitting an Ipatov preamble, there is a possibility that CAM_A will not be locked on to. However, this risk should be reduced due to: [0184] Pre-selection of the best anchor. [0185] Extra TX PWR available. [0186] No interference from SLAVE anchor.

[0187] If the response fails, the car can restart the process with swapped anchors.

Example Scenario

[0188] An example scenario is illustrated in FIG. 31, showing the user 3102 with keyfob 3104 facing away from the vehicle 3106. The vehicle includes multiple anchors as previously described. Anchor1 has a weak first path (due to the user's intervening body). Anchor2 has a weak first path but strong multipath due to a reflector 3112 (attenuation e.g., due to path obstruction is shown as thick black bars across the signal paths).

[0189] The following happens in this scenario. Both Anchor1 and Anchor2 see the fob's packet and respond. The fob 3104 latches onto Anchor2 because of the strong multipath and sees Anchor's SFD and data. The time of arrival at Anchor2, ToA.sub.2, is communicated back to fob in the data packet. DecaCyphers arrive at approximately the same time, but Anchor1's cypher arrives slightly earlier because (a) it receives the fob's initiation packet slightly earlier and (b) Anchor1's cypher

has less distance to travel. This results in a first path being present in the fob's CIR estimate coming from Anchor1 slightly earlier than the FP from Anchor1. The CIR estimate will look approximately as illustrated in FIG. 32.

[0190] The time of arrival of the packet from the fob at Anchor2, ToA.sub.2, will be the time of sending of the fob packet, ToS, plus the time of flight from the fob to Anchor2, Tp.sub.A2.

$$[00007] \text{ToA}_2 = \text{ToS} + \text{Tp}_{A2}$$

[0191] Similarly:

$$[00008] \text{ToA}_1 = \text{ToS} + \text{Tp}_{A1}$$

[0192] The Anchor will then respond after waiting for a known response delay time (Tdel). The transmission and response are illustrated in FIG. 33.

[0193] The time of arrival at the fob of the first path from Anchor2, Tfp.sub.2, will be:

$$[00009] \text{Tfp}_2 = \text{TOA}_2 + \text{Tdel}_2 + \text{Tp}_{A2}$$

[0194] Where Tdel.sub.2 is the time delay implemented in Anchor 2. This is a pre-agreed delay, but is counted with Anchor's clock which is not generally an ideally accurate clock, but is offset by a small fraction, ϵ .sub.2, generally of the order of 0-20 ppm or $0-20 \times 10^{-6}$.

$$[00010] \text{Tdel}_2 = (\text{Tdel} + \epsilon_2)(1 + \epsilon_2) = \text{Tdel}(1 + \epsilon_2) + \epsilon_2$$

[0195] A small amount of time, δ .sub.2, will be added because the transmitter only has a finite amount of control over the exact response time.

[0196] Similarly, the time of arrival at the fob of the first path from Anchor1, Tfp.sub.1, will be:

$$[00011] \text{Tfpi} = \text{TOA}_1 + \text{Tdel}_1 + \text{Tp}_{A1}$$

[0197] Substituting for ToA.sub.1:

$$[00012] \text{Tfpi} = \text{ToS} + \text{Tp}_{A1} + \text{Tdel}_1 + \text{Tp}_{A1}$$

[0198] Where $\text{Tdel.sub.1} = (\text{Tdel} + d_i)(1 + e_i) = \text{Tdel}(1 + e_i)$; the time delay implemented in Anchor 1.

[0199] The fob measures the CFO of the packet from Anchor 2 and then calculates the time of flight by subtracting the ToS and, because the message came from Anchor2, Tdel.sub.2, as:

$$[00013] \text{Tp}_{2\text{-est}} = (\text{Tfpi} - \text{ToS} - \text{Tdel}_2) / 2$$

[0200] Substitute for Tfp.sub.1:

$$[00014] = (\text{ToS} + \text{TOF}_{A1} + \text{Tdel}_1 + \text{Tp}_{A1} - \text{ToS} - \text{Tdel}_2) / 2$$

[0201] Substitute for Tde and Tdel.sub.2:

$$[00015] = (\text{ToS} + \text{TOF}_{A1} + \text{Tdel} + i\text{Tdel} + d_i + \text{Tp}_A \text{ rToS} - \text{Tdel} - 8_2 \text{Tdel} - 6_2) / 2$$

$$= (2\text{Tp}_{A1} + i\text{Tdel} + d_i - \epsilon_2 \text{Tdel} - 6_2) / 2 \quad \text{Tp}_{2\text{est}} = \text{Tp}_{Ai} + \text{Tdel}(\epsilon_1 - \epsilon_2) / 2 + (d_c - d_2) / 2$$

[0202] In the Decawave DW series ICs the transmitter can only send packets on an 8 ns boundary, this means that d.sub.c and d.sub.2 can take any value from 0 to 8 ns so (d.sub.c-d.sub.2)/2 can vary from -4 ns to 4 ns. Note that if this error is positive so that it causes the path to move further away, the farthest it can move is to the position of Tfp.sub.2. As a result, a nearer anchor can never seem further away than a farther anchor.

[0203] Tdel, is the time interval between receiving the SFD marker and sending an SFD marker. The turnaround time needs to allow the receiver to read the received packet and insert the new send time in the transmit packet so if 135 μ s is allowed for this plus and the length of the packet itself is 165 ps, then Tdel will be about 300 μ s.

[0204] If the grid shifting method discussed above is implemented (adjusting transmission time from fixed 8 ns grid by fine control of the clock frequency), then the transmitter can start transmission at any time it desires, i.e., making $d_i=0$, d.sub.2=0. In this case, the final equation will reduce to:

$$[00016] \text{Tp}_{2\text{est}} = \text{Tp}_A + \text{Tdel}(\epsilon_1 - \epsilon_2) / 2$$

[0205] If the CAMs all use almost identical clock offsets, then $b_i=e$.sub.2.

$$\text{Tp.sub.2_est} \approx \text{Tp.sub.A1}$$

[0206] So even though we received the packet from Anchor **2**, we get the time of flight to Anchor **1**, which is what we want.

How CFO Affects the Amplitude of a Path

[0207] The larger the CFO difference in the two packets, the lower the accumulated amplitude of the path. The accumulator holds the sum of a rotating first path, i.e., the sum of an exponential rotating at the carrier offset frequency. For some offsets, the path will rotate exactly one cycle over the course of the preamble. In this instance, all of the components will sum to zero. Approximating this sum by an integral we have:

$$[00017] \int_0^T e^{j\omega t} e^{jf} dt = -je^{jf} \frac{e^{j\omega T} - 1}{\omega}$$

[0208] Where ω is the carrier offset frequency, T is the length of the cypher and f is the initial phase offset. For a length 64 cypher in channel 5 (6.5 GHz) that will appear as depicted in FIG. **34**, showing a plot of path amplitude vs CFO. Notice that the magnitude is independent of the starting phase.

[0209] A zoomed path amplitude vs CFO in channel 5 is illustrated in FIG. **35**.

[0210] Up to about ± 1 ppm there is less than 3 dB attenuation and beyond ± 6.3 ppm the attenuation is more than 20 dB.

[0211] For channel 2 (4 GHz) the response is wider, as shown in FIG. **36** (path amplitude vs CFO in channel 2). For channel 9 (8 GHz), the response is narrower as shown in FIG. **37** (Path amplitude vs CFO in channel 9).

[0212] The DW1100 has the ability to adjust its clock with an accuracy of about 0.5 ppm. In order that all the first paths are visible when using simultaneous responses in channel 5, it is preferred that all the anchors' clocks are adjusted to be about 2 ppm lower than the keyfob's frequency. This will ensure the paths are not attenuated by more than about 1 dB.

Further Embodiments

[0213] In many of the above embodiments the cypher fields were the same and time-aligned.

[0214] However, in other embodiments, each responding transceiver ("CAM") sends its own (different) cypher preamble with a different delay. An example is shown in FIG. **38**. Here, the frame sent by each transceiver includes a respective different cypher field #1-#4 (corresponding to the "Decacypher" field shown in earlier embodiments).

[0215] This allows the receiver to acquire multiple distance measurements—e.g., one for each responding transceiver. To enable this, the size of the gap after each payload is not simply $N \times 128$ ns as before, but $N \times 128$ ns + $M \times (\text{cypher_length} + \text{extra gaps separating cyphers})$. The extra gap separating cyphers could be zero (as depicted in the FIG. **38** example) or greater than zero to provide greater separation (in alternative examples the cyphers could be staggered and partially overlapping).

[0216] In a further variation, multiple cypher preambles may be included in each transceiver's frame. In a specific example, as depicted in FIG. **39**, there are two cyphers (#1 and #2) in each frame. Cypher #1 is used for secure distance measurement to the nearest device (e.g., using the previously described techniques), and all cypher #1 segments are time-aligned. Cypher #2 is staggered and is intended to produce multiple peaks after correlation/accumulation. These peaks will reveal individual distances of responding transceivers. Based on these multiple distance measurements, multilateration and/or triangulation can be used to find the location (e.g., distance and direction) of the keyfob relative to the vehicle (or vice versa). Staggering of the second cypher field is achieved by a variable transmission gap between Cypher #1 and Cypher #2 (in this example 1024 ns + $n \times 128$ ns, for n ranging from 0 to 3).

[0217] Distances derived from cypher #2 may not be secure because cypher #2 may become known to an attacker after the first transmission from transceiver "CAM A". Cypher #1 may optionally be skipped; in that case only the distance measurement for transceiver "CAM A" would be secure; the rest (B,C,D) could potentially be attacked.

[0218] FIGS. **40** and **41** show alternative variations of the FIG. **39** embodiment using two cypher

preambles per frame, but with different locations of the payload. In FIG. 39, the payload precedes both cypher fields, whereas in FIG. 40 the payload is placed between the cypher fields and in FIG. 41 the payload is appended after the second cypher field, at the end of the frame.

[0219] It will be understood that the present invention has been described above purely by way of example, and modification of detail can be made within the scope of the invention.

[0220] While described in some cases as different embodiments, any or all of the variations described throughout this disclosure (for example different described organizations of frames, cypher fields, etc.) may be provided as different operating modes within a single device or system.

[0221] Various embodiments of the invention described above may provide some or all of the following features and advantages.

[0222] Embodiments may provide for the transmission of multiple packets in parallel rather than serially. This can reduce the time the receiver needs to stay active (power saving).

[0223] Various different schemes and combinations are possible. For example, multiple transmissions' start times (first Ipatov preamble symbol) can be offset by variable delays (in the examples above typically between 0 and 2000 ns but other delays can be used). What is common to many or all of the described schemes is that start of the cypher preamble is aligned across all transmissions.

[0224] Embodiments may enable calculating localization based on a single simul-packet (where the term "single simul-packet" refers to the multiple versions of a response frame transmitted in parallel). Due to variable offsets, there are multiple separated peaks in the Ipatov accumulator. This enables calculating the distances from each lock/anchor to keyfob. While multiple Ipatov-based distances may not be secure, the shortest Ipatov distance should generally match the DecaCypher-based distance. This can provide one secure distance measurement and three less secure distance measurements. Therefore localization based on such triangulation would still be reasonably secure. When the secure distance measurement (based on DecaCypher) matches the shortest of the Ipatov-based measurements, the packet can therefore be considered reliable/authentic; otherwise, the packet can be discarded and/or not utilized for distance determination.

[0225] Embodiments may implement trimming of clock crystals on responding anchor devices, so that the keyfob (the receiver of the final message) would be able to see all peaks in its accumulators. If CFOs between anchors differ by more than 1-2 ppm, then the keyfob will only see the peak of the anchor it has locked on to. There won't be any extra peaks from other anchors at all (or it will be strongly attenuated). To get all CFO's to be close to each other, all anchors listen to initial message from the keyfob, check the CFO calculated during reception and then trim their crystals to all respond at exactly the same frequency (for example 2 ppm from the keyfob's clock). Since the keyfob will know the targeted CFO (-2 ppm in this example), it can converge its carrier recovery algorithm faster (by pre-initializing carrier integrator to -2 ppm). That should allow for shortening of the Ipatov preambles.

[0226] Embodiments may implement compensation in the transmitting device, for the transmitter grid, by de-trimming the clock crystal for certain period like 200 ps, (thus speeding up time measurement or slowing it down), until it adjusts its timing offset to the required delay. This will allow the transmitter to effectively transmit at any time, rather than being limited to the 8 ns transmission grid.

[0227] Variations of the main scheme, mainly intended to shorten packet lengths in specific conditions can be provided. For example, on future chips it may be possible to configure any delay inside the message, in which case it will not be necessary to vary the number of Ipatov/data symbols. If localization is not needed and all data payloads can be identical then it is possible to implement the scheme with 0 ns offsets. Such packets are much shorter than default. This can also provide the advantage that all the anchors can transmit the packet so there is no risk that something causes no packet to be received because of a fault in the return channel from a favored preselected transmitter.

[0228] If a data payload is not needed, it can be muted and transmitted at the end (or not transmitted at all), saving power. More generally, in some variants, certain portions of the frame, e.g., Ipatov, SFD and/or data payload, are muted on some anchors and only selected anchors transmit these parts of the frame. However, all anchors transmit the cypher preamble (DecaCypher). This allows the total transmit power budget to be optimized. The anchors which transmit the Ipatov preamble can be pre-selected using various methods, e.g., based on a previous message from the keyfob or based on other transmission methods (Bluetooth, etc.). Generally, only anchors considered strongest may be selected to transmit the synchronization Ipatov preamble sequence.

[0229] The described processes or parts thereof may be implemented using one or more computer programs, stored in one or more computer-readable media, for execution by a suitable data processing device (e.g., a controller as depicted in FIG. 3A). In some embodiments, the methods may be implemented using a combination of hardware and software, but hardware-only embodiments are also possible.

Claims

1. A device for wireless communication, the device comprising: a transceiver configured to: receive a message from a second device; at least one processor communicatively coupled to the transceiver, the processor configured to: generate a response message, wherein the response message comprises a SYNC portion, a start frame delimiter (SFD) portion, a physical layer header (PHR) portion, a payload portion, a variable transmission gap, and a cipher portion; wherein the PHR portion signals a length of the variable transmission gap between the payload portion and the cipher portion of the packet; and the transceiver further configured to: transmit the response message to the second device.
2. The device of claim 1, wherein the processor is further configured to determine the length of the variable transmission gap such that the cipher portion of the packet occurs at a specific time offset relative to reception of the message.
3. The device of claim 1, wherein the cipher portion comprises a cryptographically generated training sequence.
4. The device of claim 1, wherein the processor is further configured to include information specifying the variable transmission gap in encrypted form in the payload portion.
5. The device of claim 1, wherein communication between the device and the second device is based on an ultra wide-band (UWB) communication protocol.
6. The device of claim 1, wherein the length of the variable transmission gap ranges from zero to 1016 ns.
7. The device of claim 1, wherein the length of the variable transmission gap is determined by multiplying a predetermined time by an integer.
8. A method in a device for wireless communication, the method comprising: receiving, at a transceiver, a message from a second device; generating a response message using one or more processors, wherein the response message comprises a SYNC portion, a start frame delimiter (SFD) portion, a physical layer header (PHR) portion, a payload portion, a variable transmission gap, and a cipher portion; wherein the PHR portion signals a length of the variable transmission gap between the payload portion and the cipher portion of the packet; and transmitting, using the transceiver, the response message to the second device.
9. The method of claim 8, further comprising determining, using the one or more processors, the length of the variable transmission gap such that the cipher portion of the packet occurs at a specific time offset relative to reception of the message.
10. The method of claim 8, wherein the cipher portion comprises a cryptographically generated training sequence.

- 11.** The method of claim 8, further comprising, including information specifying the variable transmission gap in encrypted form in the payload portion.
 - 12.** The method of claim 8, wherein communication between the device and the second device is based on an ultra wide-band (UWB) communication protocol.
 - 13.** The method of claim 8, wherein the length of the variable transmission gap ranges from zero to 1016 ns.
 - 14.** The method of claim 8, wherein the length of the variable transmission gap is determined by multiplying a predetermined time by an integer.
 - 15.** A non-transitory computer readable medium that stores computer executable instructions, wherein, in response to executing the computer executable instructions, one or more processors is configured to perform a method, the method comprising: receiving, with a transceiver, a message from a second device; generating a response message using the one or more processors, wherein the response message comprises a SYNC portion, a start frame delimiter (SFD) portion, a physical layer header (PHR) portion, a payload portion, a variable transmission gap, and a cipher portion; wherein the PHR portion signals a length of the variable transmission gap between the payload portion and the cipher portion of the packet; and transmitting, using the transceiver, the response message to the second device.
 - 16.** The non-transitory computer readable medium of claim 15, wherein the cipher portion comprises a cryptographically generated training sequence.
 - 17.** The non-transitory computer readable medium of claim 15, wherein the method further comprises: including information specifying the variable transmission gap in encrypted form in the payload portion.
 - 18.** The non-transitory computer readable medium of claim 15, wherein communication between the device and the second device is based on an ultra wide-band (UWB) communication protocol.
 - 19.** The non-transitory computer readable medium of claim 15, wherein the length of the variable transmission gap ranges from zero to 1016 ns.
 - 20.** The non-transitory computer readable medium of claim 15, wherein the length of the variable transmission gap is determined by multiplying a predetermined time by an integer.
-